La cola (Queue): SplQueue

Caso práctico

La tienda de **BK Programación** envía un correo de confirmación con cada pedido. Cuando hay muchos pedidos a la vez, la página tarda en responder porque espera a que se envíe el correo. **Juan** propone que la web solo **apunte** el correo en una **cola** y responda enseguida, y que otro proceso los vaya enviando **en el orden en que llegaron**.

1.- Qué es una cola

Una **cola** (*queue*) es un ADT en el que los elementos **entran por un extremo** (el *final*) y **salen por el otro** (el *frente*). El primero que entra es el primero que sale: **FIFO** (*First In, First Out*), como en la cola de la panadería.

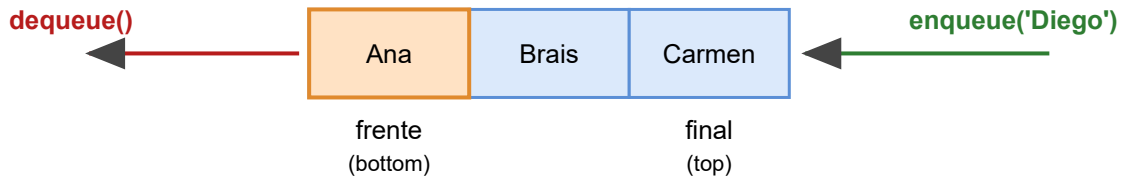


Figura 1. Una cola: se entra por el final y se sale por el frente

Operación	Qué hace	En SplQueue	Coste
enqueue	Añadir al final	`enqueue($x)`, `push($x)` o `$cola[] = $x`	O(1)
dequeue	Sacar el del frente y devolverlo	`dequeue()` (o `shift()`)	O(1)
front / *peek*	Ver el del frente sin sacarlo	`bottom()`	O(1)
back	Ver el último que llegó	`top()`	O(1)
isEmpty, *size*	¿Vacía? / número de elementos	`isEmpty()`, `count($cola)`	O(1)

2.- La clase SplQueue

`SplQueue` hereda de `SplDoublyLinkedList` con el modo de recorrido fijado en FIFO.

```php
<?php
$impresora = new SplQueue();
$impresora->enqueue('matricula-ana.pdf');
$impresora->enqueue('notas-1DAW.pdf');
$impresora[] = 'horario.pdf';                 // también añade al final

echo $impresora->dequeue();                   // "matricula-ana.pdf" (el primero que llegó)
echo $impresora->bottom();                    // "notas-1DAW.pdf": el siguiente
echo count($impresora);                       // 2

while (!$impresora->isEmpty()) {              // la forma habitual de vaciar una cola
    imprimir($impresora->dequeue());
}
```

Debes conocer

Nombres confusos. Como `SplQueue` es una lista en la que se mete por "arriba", `bottom()` es el **frente** (el siguiente en salir) y `top()` es el **final** (el último que llegó). Y, como en la pila, con la cola vacía `dequeue()`, `bottom()` y `top()` lanzan `RuntimeException`.

Recorrer. `foreach` va del frente al final sin sacar a nadie (claves 0, 1, 2...), y `$cola[0]` es el frente. Con `setIteratorMode(SplDoublyLinkedList::IT_MODE_FIFO | SplDoublyLinkedList::IT_MODE_DELETE)`, `foreach` va sacando cada elemento. El sentido FIFO no se puede cambiar.

3.- ¿Por qué no un array?

Con un array, añadir al final (`$a[] = $x`) es rápido, pero **sacar el primero con `array_shift()` es O(n)**: PHP tiene que renumerar todos los demás elementos. Con muchos elementos la diferencia es enorme: en el ejemplo `e01-SplQueue.php`, vaciar una cola de 20 000 elementos tarda unos 150 ms con `array_shift` y menos de 1 ms con `SplQueue::dequeue()`.

Recomendación

Si vas a sacar elementos por el principio de una colección grande de forma repetida, no uses `array_shift`: usa `SplQueue` (o un índice que avance por el array sin borrar nada).

4.- Aplicaciones típicas

Problema	Por qué una cola
Trabajos pendientes (correos, imágenes que redimensionar, pedidos)	Se atienden en el orden de llegada; los que fallan pueden volver al final
Colas de impresión, turnos de atención	Primero en llegar, primero en ser atendido
Planificación de procesos (*round-robin*)	Cada proceso trabaja un poco y vuelve al final
Búsqueda en anchura (BFS): camino más corto, grados de separación en una red social	Se exploran primero los vecinos más cercanos
Buffers ("los N últimos mensajes"), medias móviles	Al llegar uno nuevo sale el más antiguo

La búsqueda en anchura es la aplicación más importante: al usar una cola, se visitan primero todas las casillas (o personas) a distancia 1, luego las de distancia 2... por lo que la primera vez que se llega al destino, el camino es el **más corto**. Con una pila, en cambio, se encontraría un camino cualquiera.

```php
<?php
function distanciaMinima(array $mapa, array $inicio): ?int     // '#' = muro, 'C' = cliente
{
    $cola = new SplQueue();
    $cola->enqueue([...$inicio, 0]);                           // fila, columna, distancia
    $visitadas = [implode(',', $inicio) => true];
    while (!$cola->isEmpty()) {
        [$f, $c, $d] = $cola->dequeue();
        if ($mapa[$f][$c] === 'C') {
            return $d;
        }
        foreach ([[0, 1], [1, 0], [0, -1], [-1, 0]] as [$df, $dc]) {
            [$nf, $nc] = [$f + $df, $c + $dc];
            if ($nf >= 0 && $nc >= 0 && isset($mapa[$nf][$nc]) && $mapa[$nf][$nc] !== '#' &&
!isset($visitadas["$nf,$nc"])) {
                $visitadas["$nf,$nc"] = true;
                $cola->enqueue([$nf, $nc, $d + 1]);
            }
        }
    }
    return null;                                               // no hay camino
}
```

Debes conocer

Fíjate en `$nc >= 0`: en PHP, `$texto[-1]` es el **último** carácter de un string (y `$array[-1]` puede existir), así que sin esa comprobación el recorrido "atravesaría" el borde del mapa. Es un error muy habitual al trabajar con cuadrículas.

5.- Ejercicio resuelto: cola de envío con reintentos

Ejercicio resuelto

La tienda guarda en una cola los correos que hay que enviar. Si un envío falla, el correo vuelve al final para reintentarlo, como mucho 3 veces.

```
<?php
$correos = new SplQueue();
foreach (['ana@example.com', 'brais@caido.test', 'carmen@example.com'] as $destino) {
    $correos->enqueue(['para' => $destino, 'intentos' => 0]);
}
$registro = [];
while (!$correos->isEmpty()) {
    $trabajo = $correos->dequeue();
    $trabajo['intentos']++;
    $ok = !str_ends_with($trabajo['para'], '.test');     // simulación: .test siempre
falla
    if ($ok) {
        $registro[] = "enviado a {$trabajo['para']} (intento {$trabajo['intentos']})";
    } elseif ($trabajo['intentos'] < 3) {
        $correos->enqueue($trabajo);                      // al final: se reintentará
después
        $registro[] = "fallo con {$trabajo['para']}, se reintentará";
    } else {
        $registro[] = "descartado {$trabajo['para']} tras 3 intentos";
    }
}
print_r($registro);
// enviado a ana / fallo con brais / enviado a carmen / fallo con brais / descartado brais
tras 3 intentos
```

Observa que el correo que falla no bloquea a los demás: vuelve al final y, mientras tanto, se envía el de Carmen. Así funcionan los sistemas de colas profesionales (RabbitMQ, Redis, las *jobs* de Laravel), aunque allí la cola se guarda fuera de PHP para que sobreviva entre peticiones.

Autoevaluación

- Se encolan 'a', 'b' y 'c' y se hace `dequeue()`. ¿Qué se obtiene? **"a": el primero en entrar.**
- En una `SplQueue`, ¿qué devuelve `bottom()`? **El frente: el siguiente que saldrá.**
- ¿Por qué `array_shift` no es buena idea para una cola grande? **Porque es O(n): renumera todos los elementos cada vez.**
- ¿Por qué la búsqueda en anchura encuentra el camino más corto? **Porque la cola hace que se visiten todos los puntos a distancia d antes que los de distancia d + 1.**

6.- Ejercicios

Fichero	Contenido
`Ejercicios Queue/e01-SplQueue.php`	La API: `enqueue`, `dequeue`, `bottom`, `top`, cola vacía, `foreach`, `IT_MODE_DELETE`, `array_shift` frente a `dequeue`
`Ejercicios Queue/e02-SplQueue.php`	Aplicaciones: cola con reintentos, *round-robin*, camino más corto (BFS), últimos N mensajes
`Ejercicios Queue/problemas/`	`e01` y `e02`: problemas y soluciones
